# PYTHON ABL
# PROGRAMS REPORT

Problem Statements • Standard Flowcharts • Pseudocode • Sample Outputs • Source Code

**Repository:** inapakollasirisha21-rgb / PYTHON-ABL

**GitHub:** https://github.com/inapakollasirisha21-rgb/PYTHON-ABL

Updated edition: flowcharts use standard symbols — oval terminators, parallelograms for input/output, rectangles for processes, diamonds for decisions, and directional arrows.

# Contents

case1.py — Student Marks, Total, Average and Result

case2.py — Attendance Eligibility

case3.py — Simple Banking Menu

case4.py — Shopping Bill Calculator

case5.py — Analyze a List of Marks

case6.py — Password Length Validation

case7.py — Basic Email Format Check

case8.py — Meal Leftover and Wastage Percentage

case9.py — Product Inventory Value

case10.py — Totals and Averages for Multiple Students

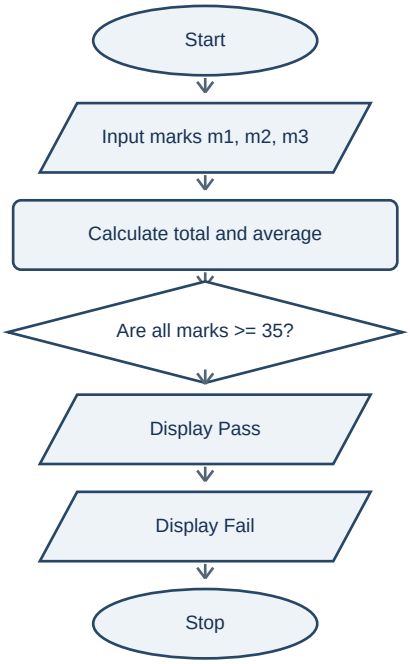
# 1. Student Marks, Total, Average and Result

**Source file:** case1.py

## Problem Statement

Read marks in three subjects, calculate total and average, and display Pass only if all three marks are at least 35; otherwise display Fail.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start
2. Read m1, m2, m3
3. total = m1 + m2 + m3; average = total / 3
4. IF all marks >= 35 THEN display Pass ELSE display Fail
5. Stop
```

## Sample Input

```
Enter marks in subject 1: 80
Enter marks in subject 2: 70
Enter marks in subject 3: 90
```

## Expected Output

```
Total: 240
Average: 80.0
Result: Pass
```

## Source Code

```
m1 = int(input("Enter marks in subject 1: "))
m2 = int(input("Enter marks in subject 2: "))
m3 = int(input("Enter marks in subject 3: "))
total = m1 + m2 + m3
avg = total / 3
print("Total:", total)
print("Average:", avg)
if m1 >= 35 and m2 >= 35 and m3 >= 35:
```

```
    print("Result: Pass")
else:
    print("Result: Fail")
```

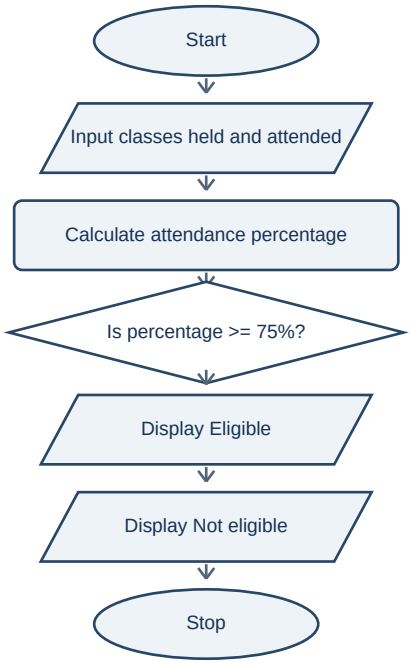
# 2. Attendance Eligibility

**Source file:** case2.py

## Problem Statement

Calculate attendance percentage from classes held and attended. Display exam eligibility when attendance is at least 75%.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start
2. Read classes held and attended
3. percentage = attended / held * 100
4. IF percentage >= 75 THEN display Eligible ELSE display Not eligible
5. Stop
```

## Sample Input

```
Classes held: 40
Classes attended: 32
```

## Expected Output

```
Attendance: 80.0 %
Eligible for exam
```

## Source Code

```
held = int(input("Classes held: "))
attended = int(input("Classes attended: "))
percentage = attended / held * 100
print("Attendance:", percentage, "%")
if percentage >= 75:
    print("Eligible for exam")
else:
    print("Not eligible for exam")
```

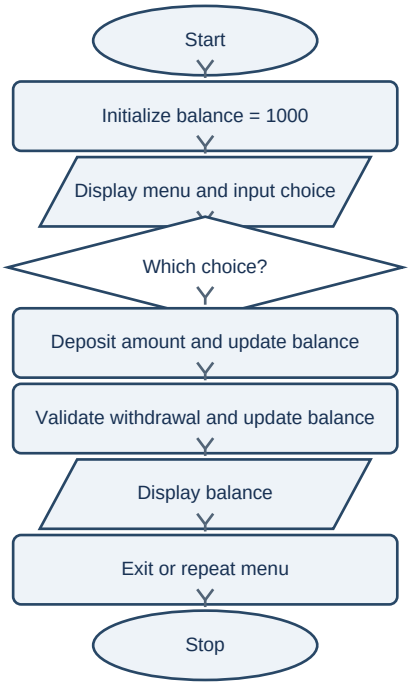
# 3. Simple Banking Menu

**Source file:** case3.py

## Problem Statement

Create a menu-driven program to deposit, withdraw, check balance, or exit. Allow withdrawal only for a positive amount not exceeding the balance.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start; balance = 1000
2. Display menu and read choice
3. IF choice = 1: deposit amount and add to balance
4. ELSE IF choice = 2: read amount; validate; subtract if valid
5. ELSE IF choice = 3: display balance
6. ELSE IF choice = 4: exit; otherwise show invalid choice
7. Repeat until exit
8. Stop
```

## Sample Input

```
Enter choice: 3
Enter choice: 1
Deposit amount: 500
Enter choice: 3
Enter choice: 4
```

## Expected Output

```
Menu displayed; Balance: 1000; after deposit, Balance: 1500; Exit.
```

## Source Code

```
balance = 1000
while True:
    print("1.Deposit 2.Withdraw 3.Balance 4.Exit")
    choice = int(input("Enter choice: "))
    if choice == 1:
```

```
    balance += int(input("Deposit amount: "))
elif choice == 2:
    amount = int(input("Withdraw amount: "))
    if amount <= balance and amount > 0:
        balance -= amount
    else:
        print("Invalid amount or insufficient balance")
elif choice == 3:
    print("Balance:", balance)
elif choice == 4:
    break
else:
    print("Invalid choice")
```

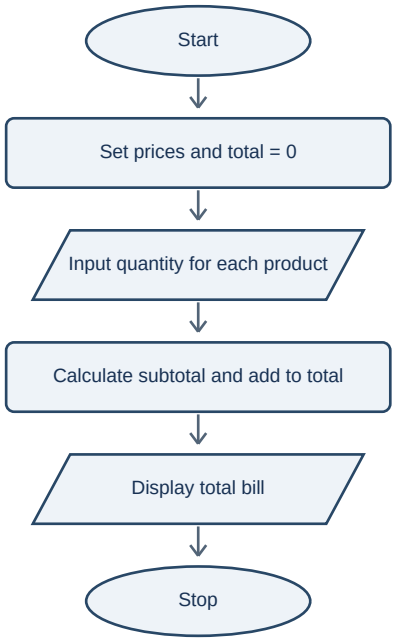
# 4. Shopping Bill Calculator

**Source file:** case4.py

## Problem Statement

Use a list of three product prices. Read quantity for each product, calculate each subtotal, and display the total bill.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

## Pseudocode

```
1. Start
2. prices = [100, 200, 150]; total = 0
3. FOR each price: read quantity; total += price * quantity
4. Display total bill
5. Stop
```

## Sample Input

```
Enter quantity: 2
Enter quantity: 1
Enter quantity: 3
```

## Expected Output

```
Total bill: 850
```

## Source Code

```
prices = [100, 200, 150]
total = 0
for price in prices:
    quantity = int(input("Enter quantity: "))
    total += price * quantity
print("Total bill:", total)
```

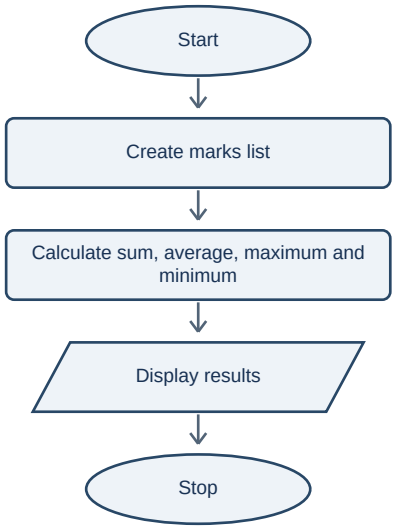
# 5. Analyze a List of Marks

**Source file:** case5.py

## Problem Statement

Calculate and display the total, average, highest, and lowest marks from a list.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

## Pseudocode

```
1. Start
2. marks = [85, 70, 92, 65, 78]
3. total = sum(marks); average = total / len(marks)
4. highest = max(marks); lowest = min(marks)
5. Display all results
6. Stop
```

## Sample Input

```
No input required.
```

## Expected Output

```
Total: 390
Average: 78.0
Highest marks: 92
Lowest marks: 65
```

## Source Code

```
marks = [85, 70, 92, 65, 78]
print("Total:", sum(marks))
print("Average:", sum(marks) / len(marks))
print("Highest marks:", max(marks))
print("Lowest marks:", min(marks))
```

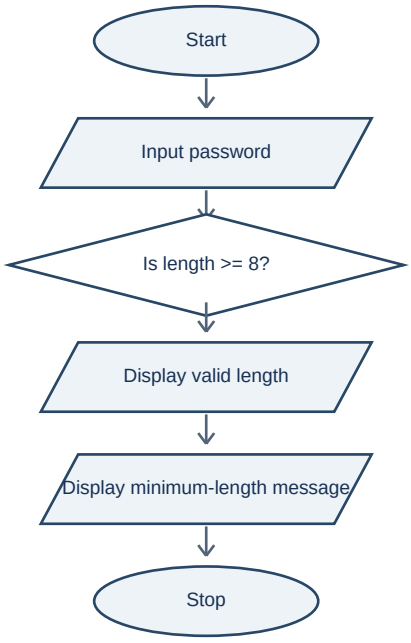
# 6. Password Length Validation

**Source file:** case6.py

## Problem Statement

Read a password and check whether it contains at least eight characters.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start
2. Read password
3. IF len(password) >= 8 THEN display valid length ELSE display minimum-length message
4. Stop
```

## Sample Input

```
Enter password: python123
```

## Expected Output

```
Password length is valid
```

## Source Code

```
password = input("Enter password: ")
if len(password) >= 8:
    print("Password length is valid")
else:
    print("Password must contain at least 8 characters")
```

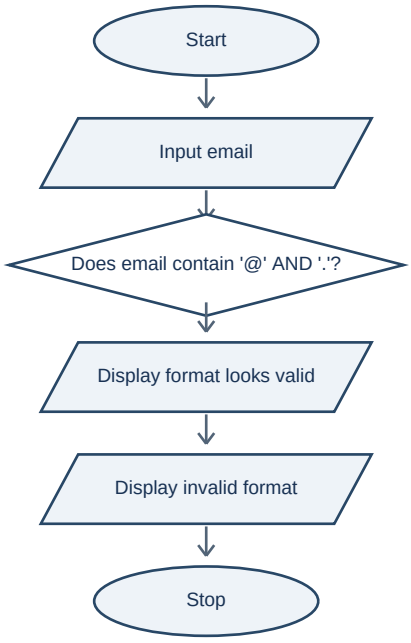
# 7. Basic Email Format Check

**Source file:** case7.py

## Problem Statement

Read an email string and check whether it contains both '@' and '.'.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start
2. Read email
3. IF '@' in email AND '.' in email THEN display valid ELSE display invalid
4. Stop
```

## Sample Input

```
Enter email: student@example.com
```

## Expected Output

```
Email format looks valid
```

## Source Code

```
email = input("Enter email: ")
if "@" in email and "." in email:
    print("Email format looks valid")
else:
    print("Invalid email format")
```

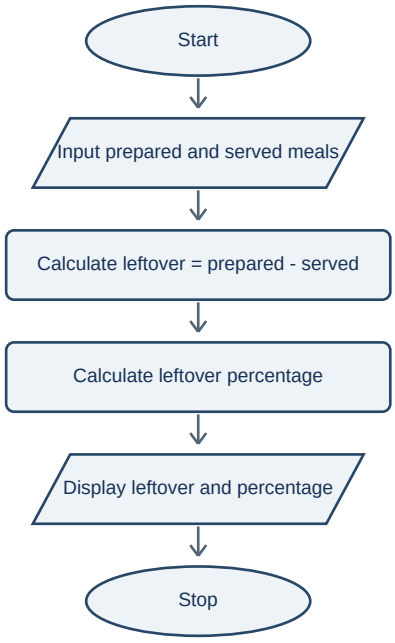
# 8. Meal Leftover and Wastage Percentage

**Source file:** case8.py

## Problem Statement

Read meals prepared and served. Calculate leftover meals and the percentage of prepared meals left over.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

## Pseudocode

```
1. Start
2. Read prepared and served meals
3. leftover = prepared - served
4. percentage = leftover / prepared * 100
5. Display leftover and percentage
6. Stop
```

## Sample Input

```
Meals prepared: 100
Meals served: 85
```

## Expected Output

```
Leftover meals: 15
Wastage percentage: 15.0 %
```

## Source Code

```
prepared = int(input("Meals prepared: "))
served = int(input("Meals served: "))
leftover = prepared - served
percentage = leftover / prepared * 100
print("Leftover meals:", leftover)
print("Wastage percentage:", percentage, "%")
```

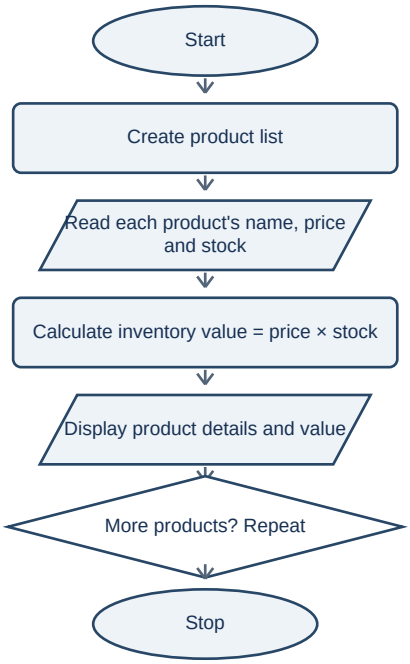
# 9. Product Inventory Value

**Source file:** case9.py

## Problem Statement

Display each product's name, price and stock quantity, and calculate inventory value as price multiplied by stock.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start
2. Create list of (name, price, stock) tuples
3. FOR each product: display details
4. inventory_value = price * stock
5. Display inventory value
6. Stop
```

## Sample Input

```
No input required.
```

## Expected Output

```
Pen: 200
Book: 500
Bag: 2500
```

## Source Code

```
products = [("Pen", 10, 20), ("Book", 50, 10),
            ("Bag", 500, 5)]
for name, price, stock in products:
    print(name, "Price:", price, "Stock:", stock)
    print("Inventory value:", price * stock)
```

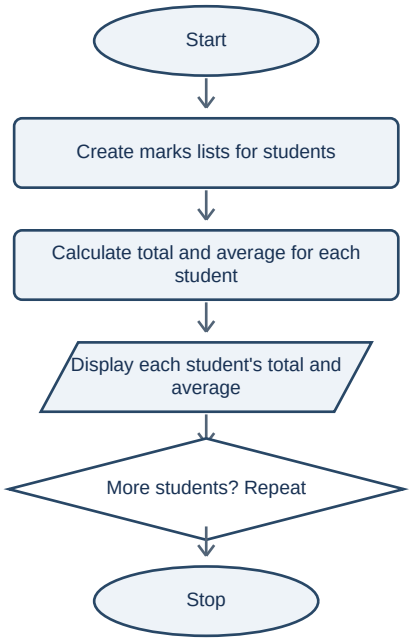
# 10. Totals and Averages for Multiple Students

**Source file:** case10.py

## Problem Statement

For three students, calculate and display total and average marks across the subjects.

## Standard Flowchart




Oval: Start/Stop  Parallelogram: Input/Output  Rectangle: Process  Diamond: Decision

**Decision branches:** Follow the Yes/No outcomes from the decision diamond to the corresponding result/action, then continue to Stop or the next loop step.

## Pseudocode

```
1. Start
2. marks = [[80,75,90], [60,85,70], [95,90,88]]
3. FOR each student's marks: total = sum(marks)
4. average = total / number of subjects
5. Display student number, total and average
6. Stop
```

## Sample Input

```
No input required.
```

## Expected Output

```
Student 1: Total 245, Average 81.66666666666667
Student 2: Total 215, Average 71.66666666666667
Student 3: Total 273, Average 91.0
```

## Source Code

```
marks = [[80, 75, 90], [60, 85, 70], [95, 90, 88]]
for i in range(len(marks)):
    total = sum(marks[i])
    average = total / len(marks[i])
    print("Student", i + 1)
    print("Total:", total)
    print("Average:", average)
```